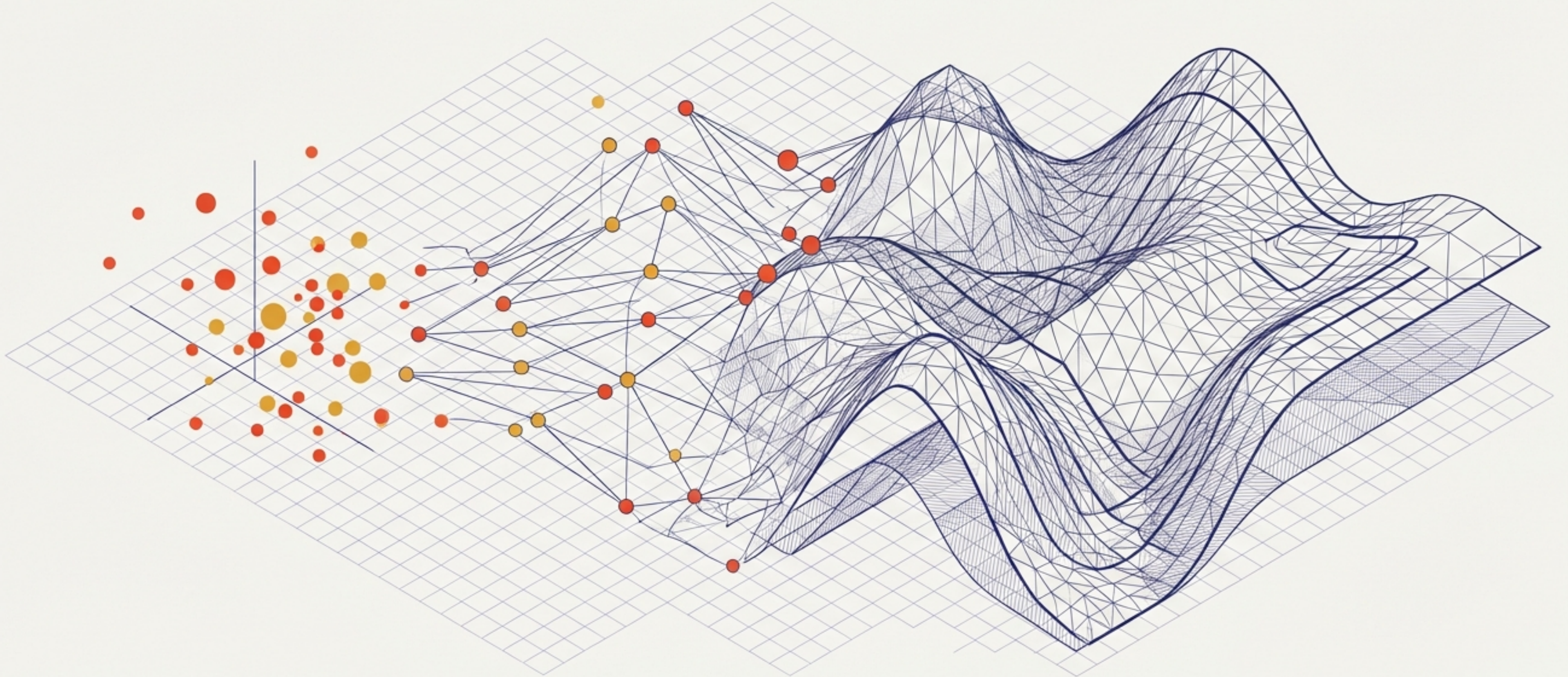


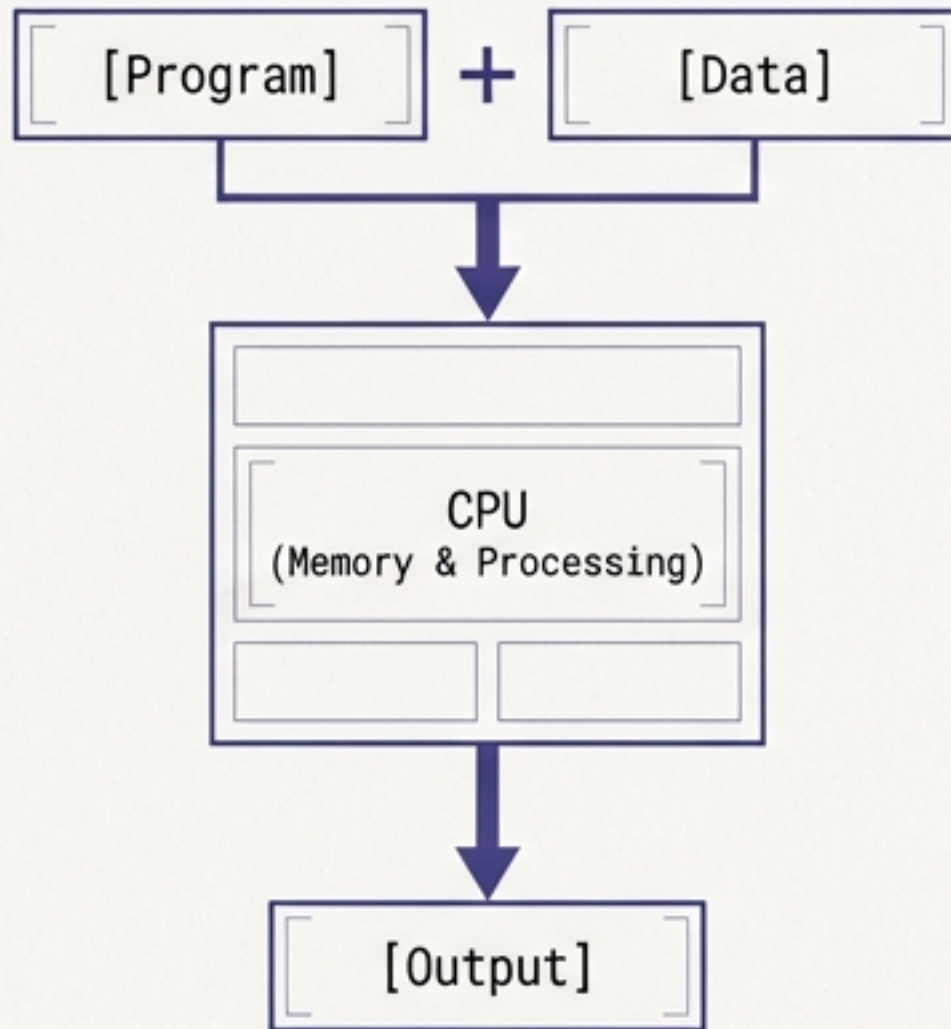
The Topological Blueprint of Deep Learning

Constructing **universal approximation** through spatial geometry and multi-layered hyperplanes.



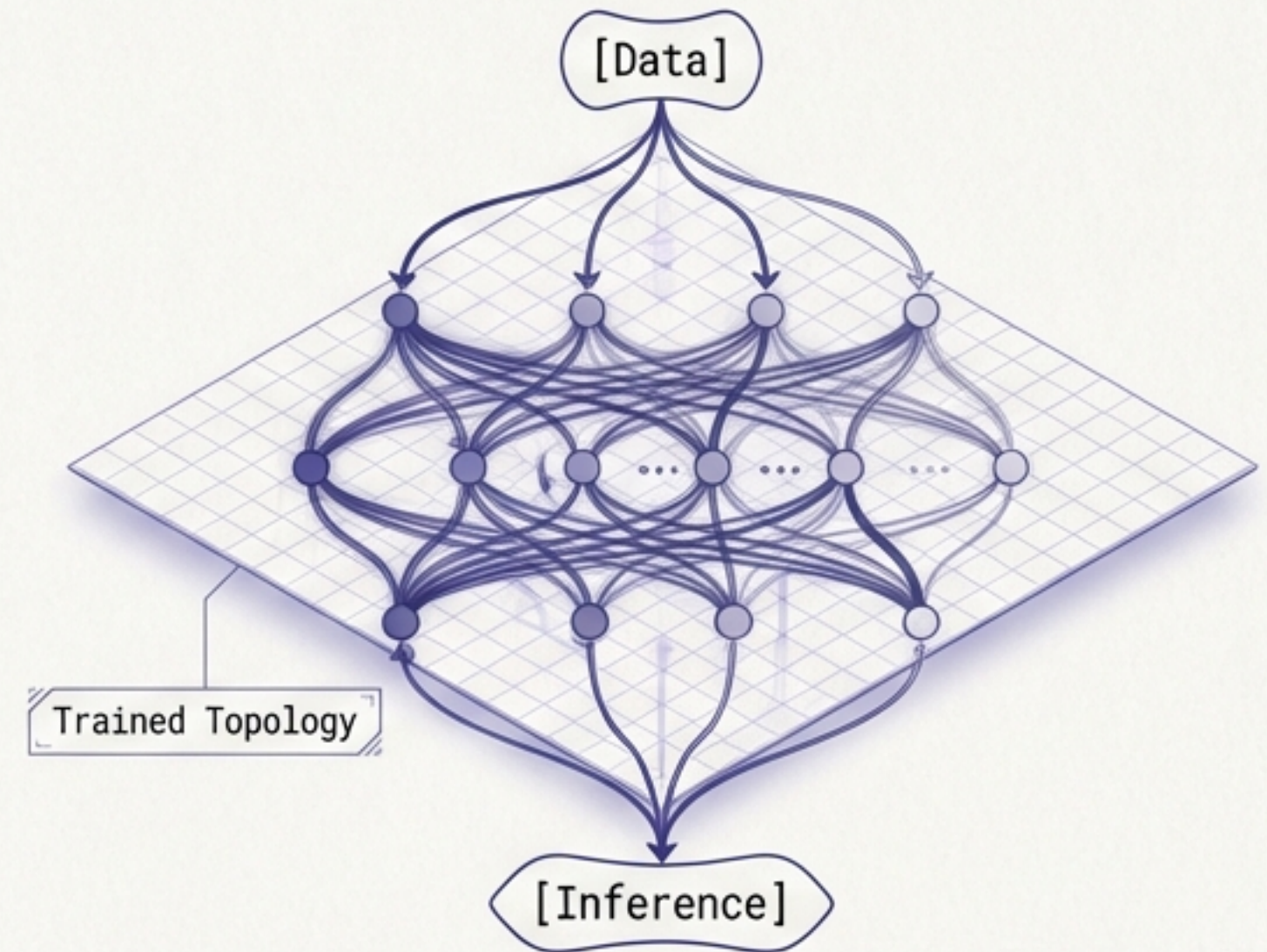
From Procedural Logic to Spatial Inference

von Neumann Architecture



- Instructions are isolated from the processor.
- Execution is linear, explicit, and pre-defined.

Neural Network Architecture



- No separation of processor and program.
- The network is the function.
- Knowledge is embedded in the mathematical structure of the connections.

The Target Function Taxonomy

Logical Functions

Definition: Binary variable evaluation (0 or 1).

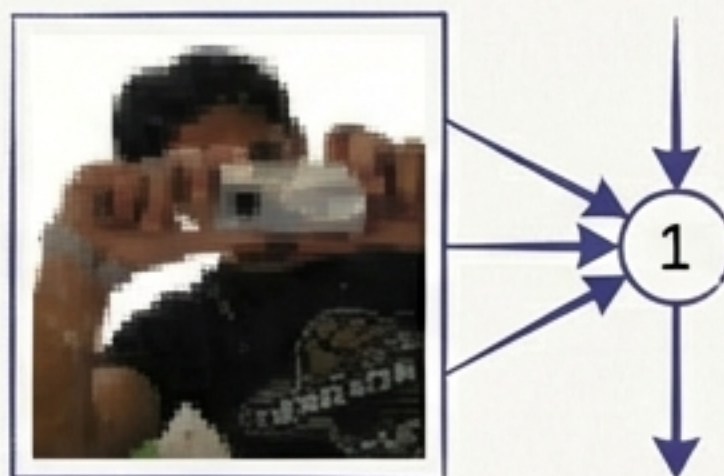
$$\vec{x} \in \{0, 1\}^d \rightarrow y \in \{0, 1\}$$

Real-World Example:
Autonomous braking
(m -out-of- n sensor trigger).

Classifiers (Categorical)

Definition: Partitioning continuous feature space into discrete classes.

$$\vec{x} \in \mathbb{R}^d \rightarrow y \in \{C_1, C_2, \dots, C_k\}$$



Real-World Example:
Discriminative face
detection.

General Functions (Continuous)

Definition: Estimating probabilities or continuous variables.

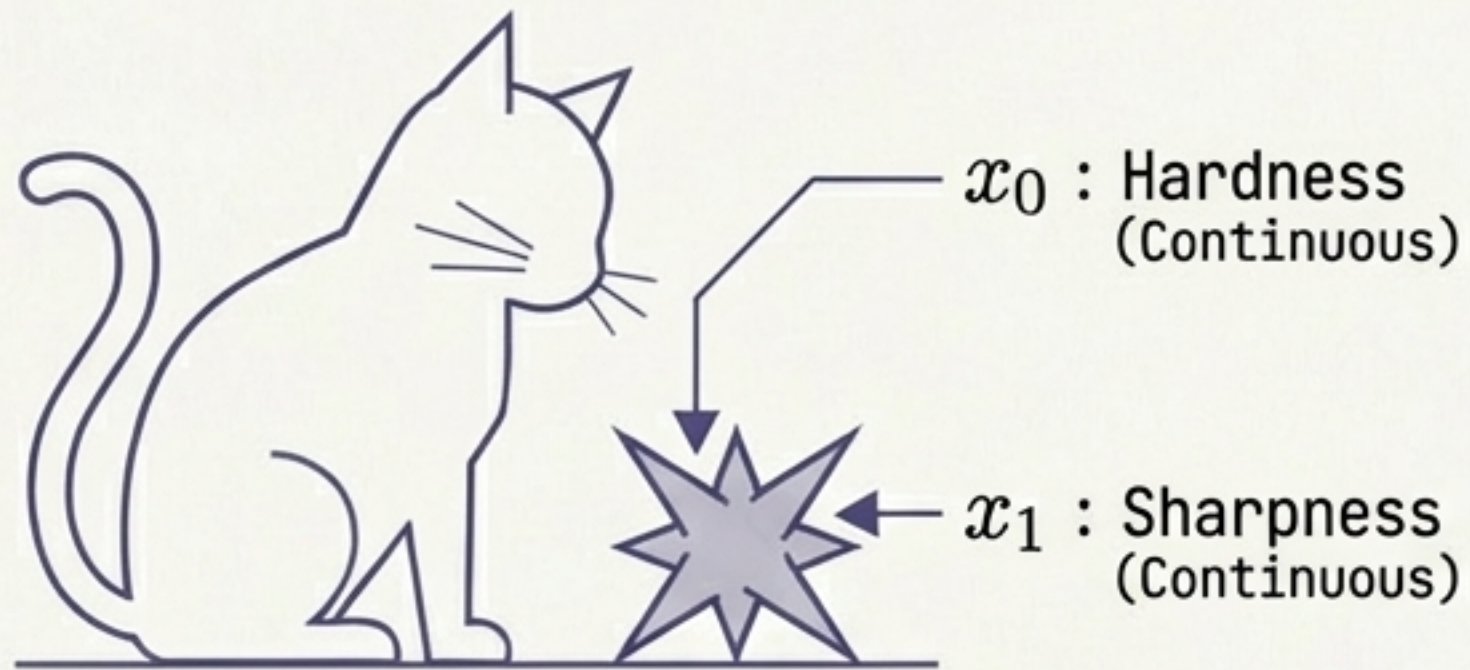
$$\vec{x} \in \mathbb{R}^d \rightarrow y \in \mathbb{R}$$

Real-World Example:
Estimating self-driving
vehicle speed based on
surrounding traffic parameters.

Vectorizing Reality: The Feature Space

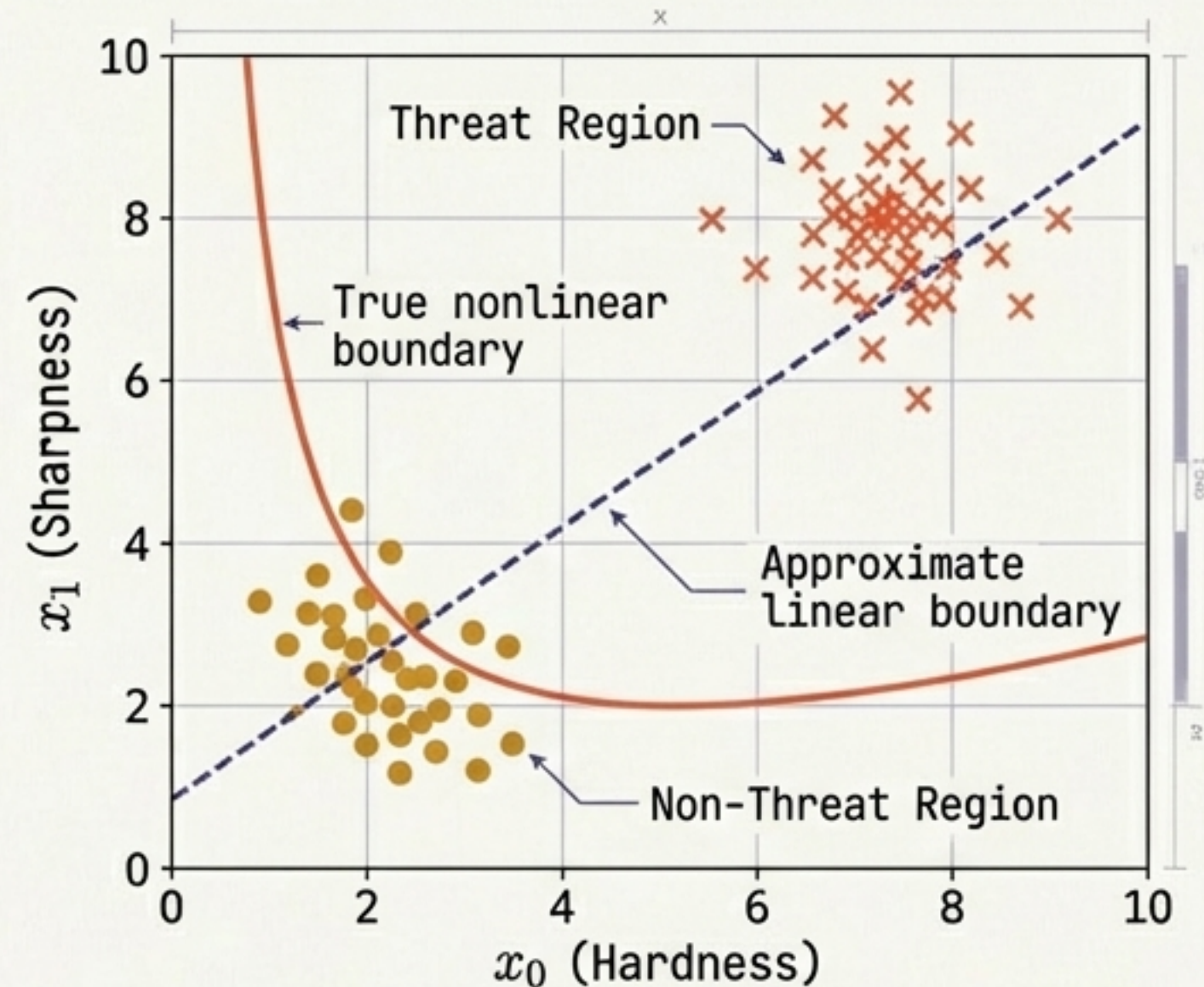
Classification is the geometric act of partitioning a multi-dimensional space into separate regions.

The Physical Threat Model



Input Vector: $\vec{x} = [x_0, x_1]$

The Mathematical Feature Space



The Primary Cut: Anatomizing the Perceptron

The Math

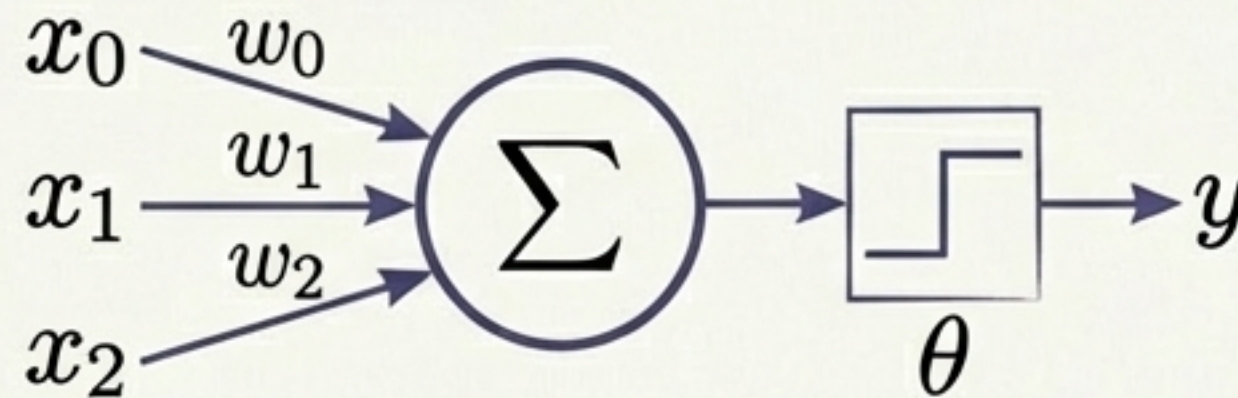
$$P(\vec{x}) = \theta(\vec{w}^T \vec{x} + b)$$

Heaviside step function

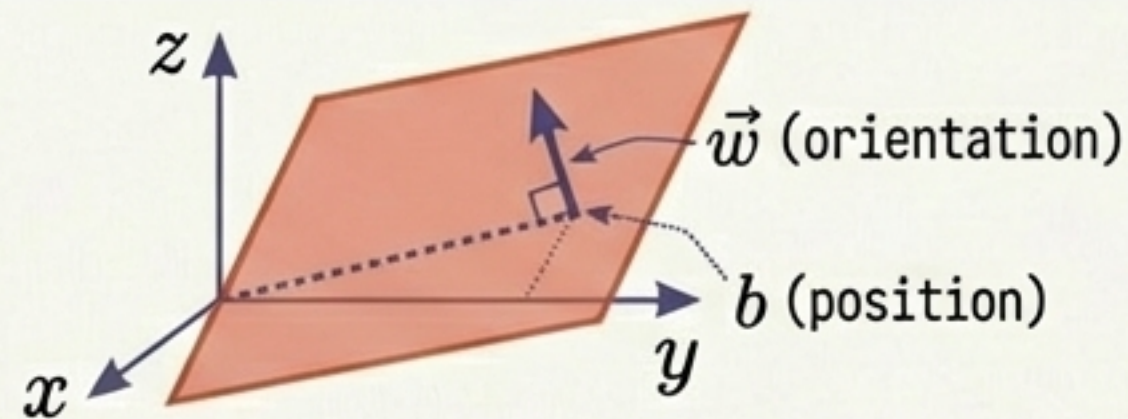
Weight vector
(Normal to the plane)

Bias
(Offset from origin)

The Architecture



The Geometry

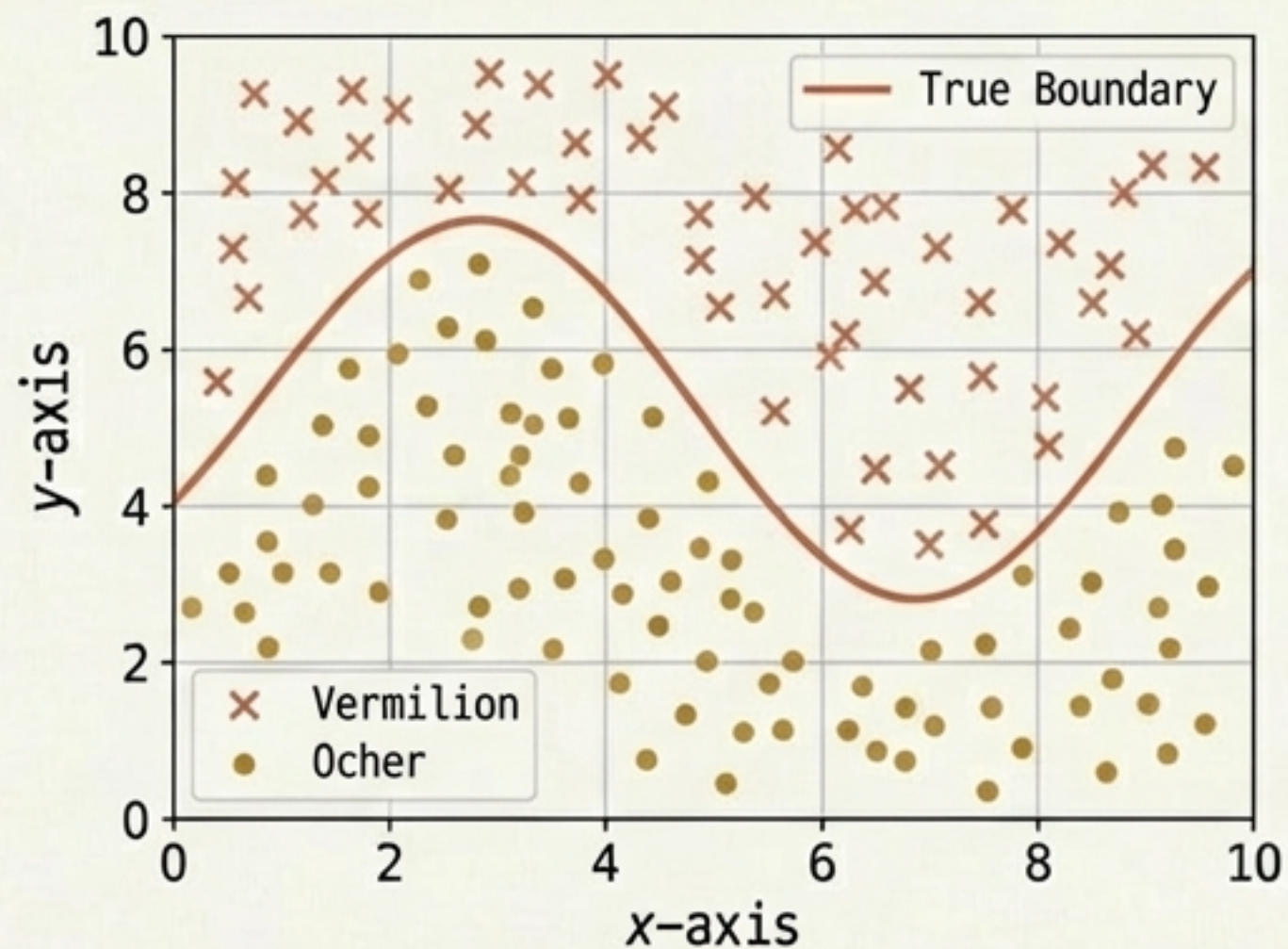


A single perceptron acts as a linear classifier, bisecting any n -dimensional space into two distinct half-spaces.

Boundary Quality & Data Distribution

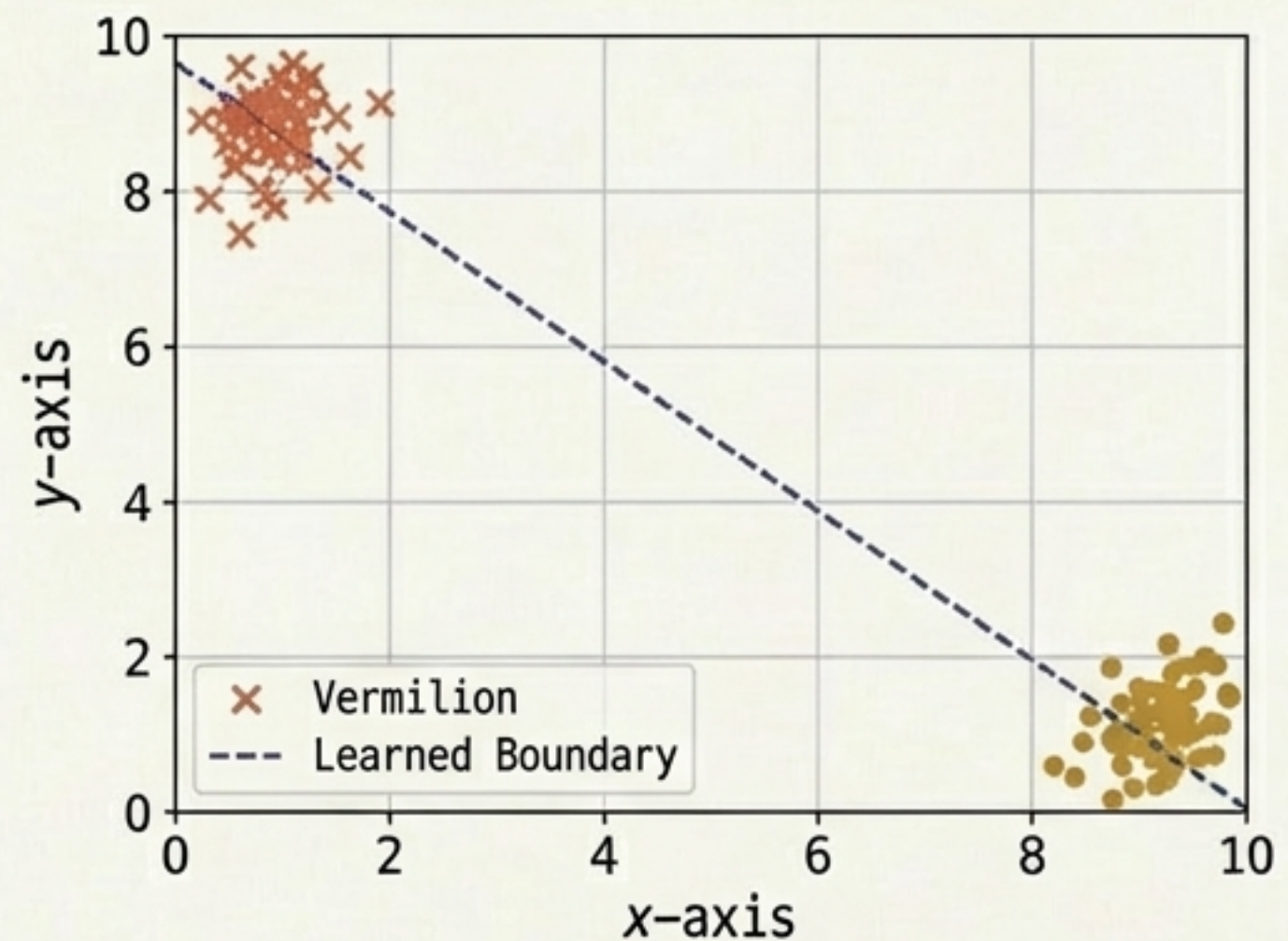
Networks do not learn the true boundary;
they learn an optimal partition of the provided samples.

Good Training Data



Sample points span the true underlying distribution. The resulting learned boundary is highly accurate.

Bad Training Data



Sample points are clustered and incomplete. The learned boundary successfully separates the clusters but entirely fails to map reality.

The Limit of Linearity

Why deep architectures are mathematically mandatory.

Logic Gate	Truth Table	Geometry	Architecture															
AND	<table><tr><th>In1</th><th>In2</th><th>Out</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	In1	In2	Out	0	0	0	0	1	0	1	0	0	1	1	1	 Linearly Separable	Single Perceptron
In1	In2	Out																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
CORRECTED	<table><tr><th>In1</th><th>In2</th><th>Out</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	In1	In2	Out	0	0	0	0	1	1	1	0	1	1	1	1	 Linearly Separable	Single Perceptron
In1	In2	Out																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
XOR	<table><tr><th>In1</th><th>In2</th><th>Out</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	In1	In2	Out	0	0	0	0	1	1	1	0	1	1	1	0	 Impossible to separate with a single straight line. Non-Separable.	Requires 2-Layer Multi-Layer Perceptron (MLP)
In1	In2	Out																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Architectural Stacking: Escaping Linearity

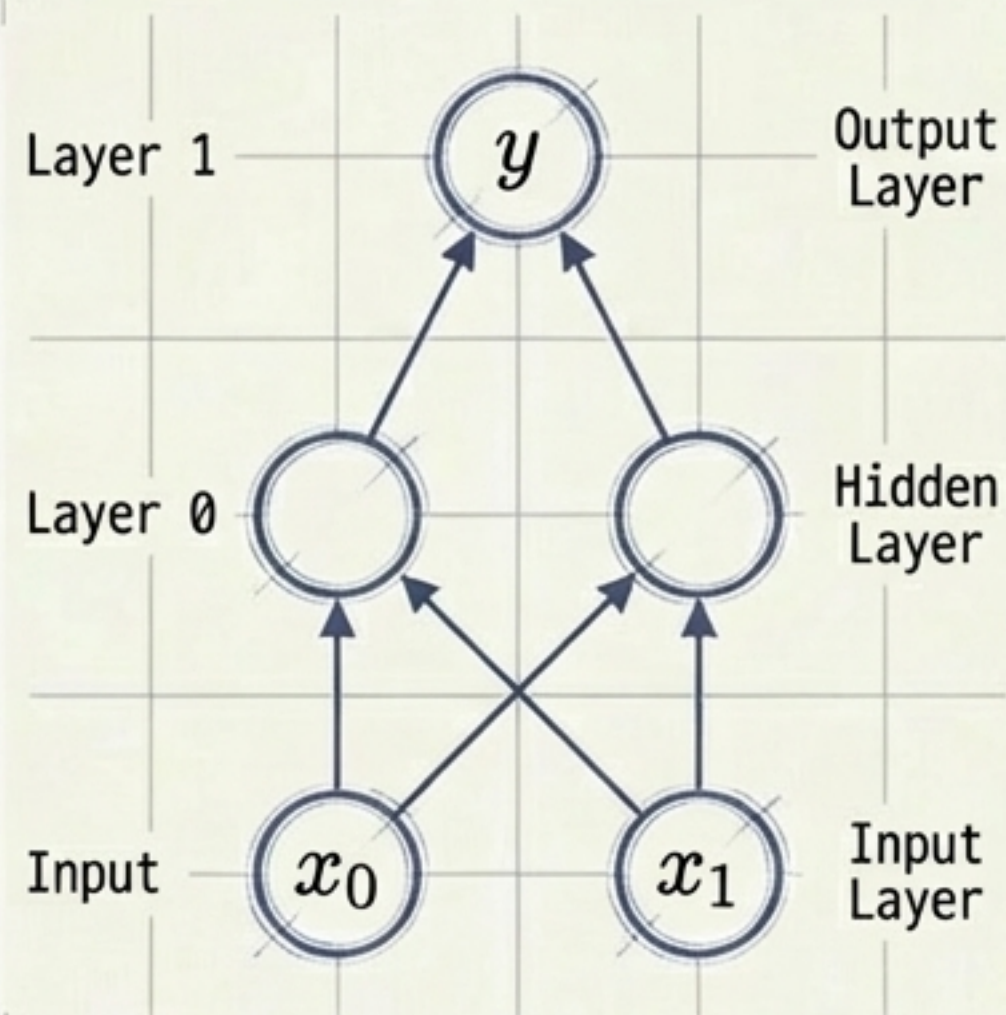
Press network Multilayers 2-layer min layer problem (MLP.

The Math

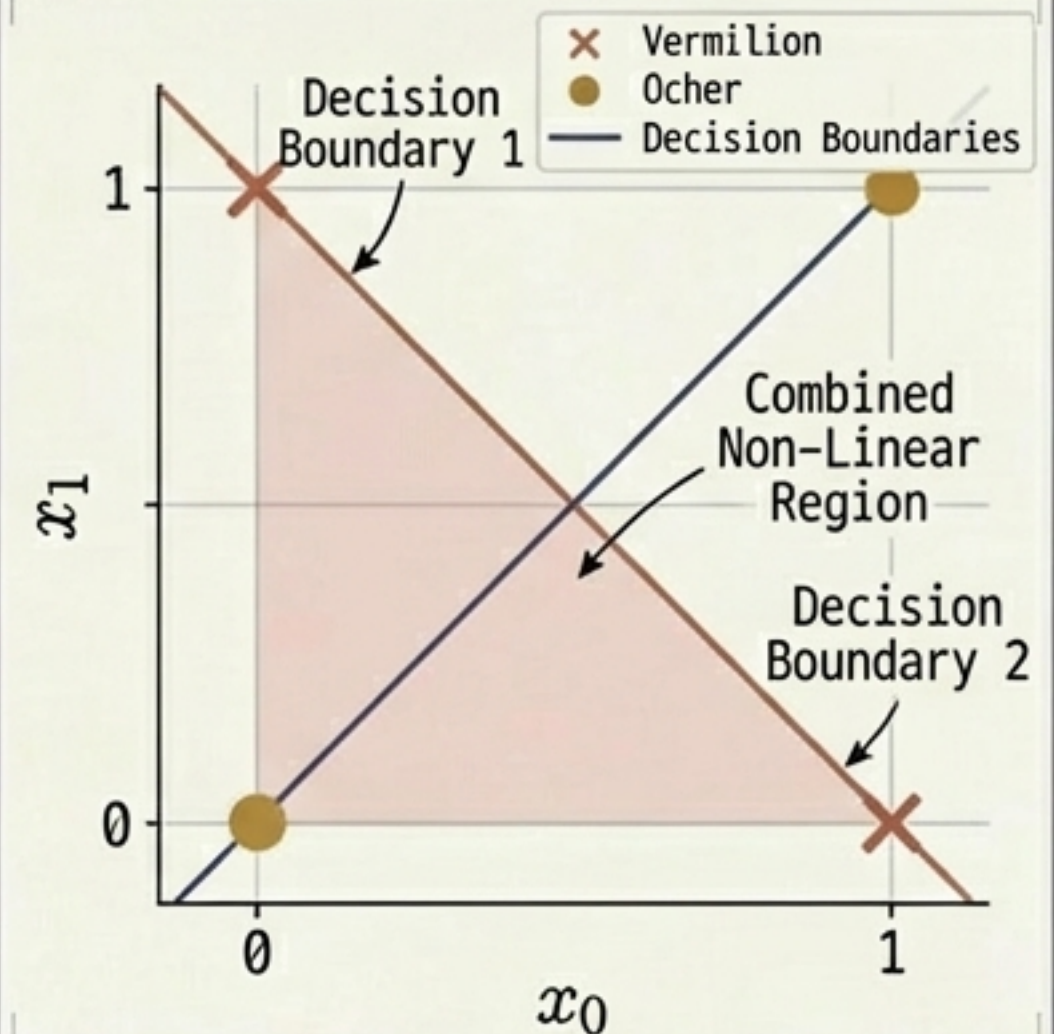
$$y = (\neg x_0 \wedge x_1) \vee (x_0 \wedge \neg x_1)$$

Logical combination of linear decision boundaries.

The Architecture



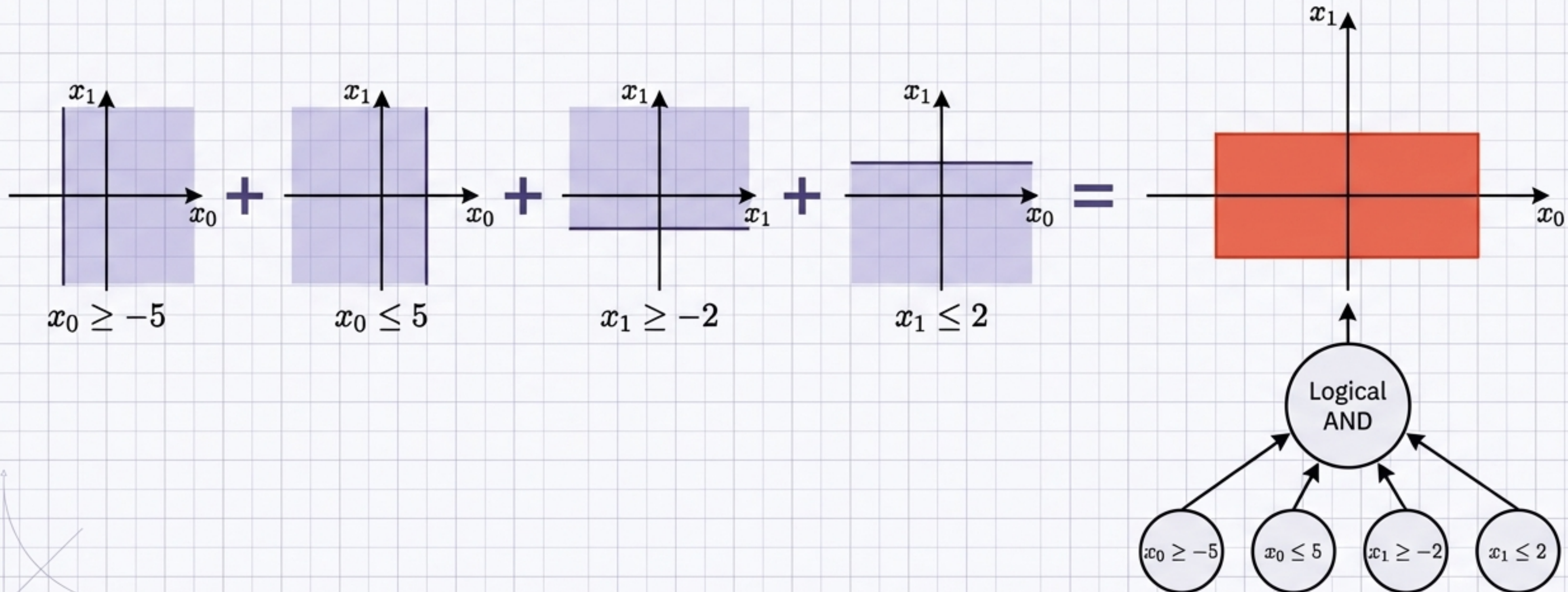
The Geometry



Layering perceptrons allows outputs of linear functions to act as inputs for the next. Hidden layers mathematically represent the combination of multiple straight cuts.

Bounding Space: The Polygon Builder

Any continuous shape can be approximated by polygons. Given enough perceptrons, an MLP can isolate an arbitrary region of feature space.



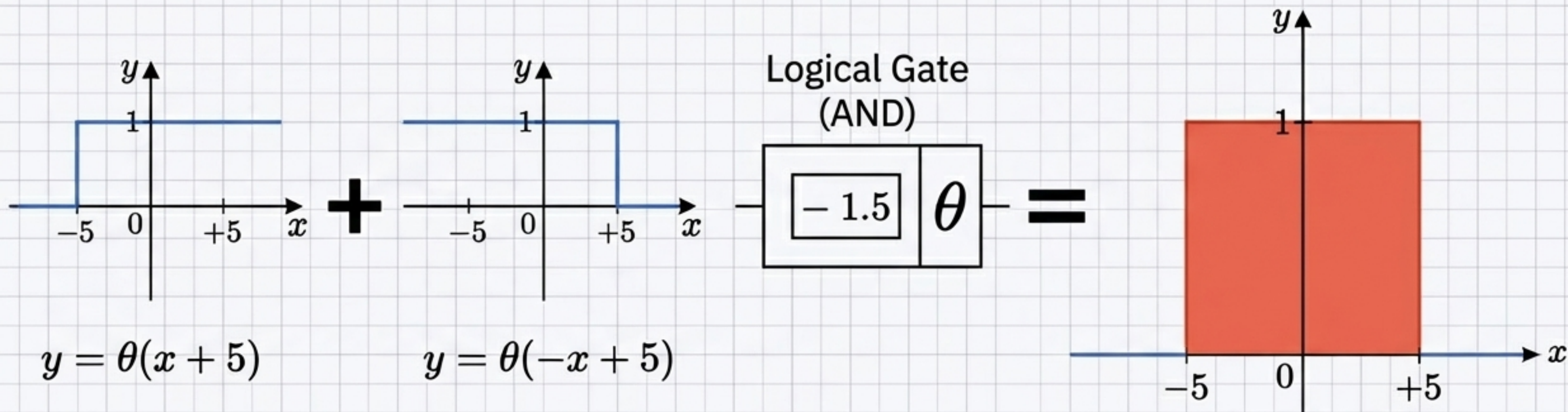
Universal Approximation: The Premise



Cybenko's Universal Approximation Theorem proves that a neural network with just one hidden layer can approximate any continuous real-world function $y = f(x)$.

The Conceptual Leap: If an MLP can generate a single, isolated "tower" in space, it can generate an infinite series of them. Summing these towers allows the network to model any arbitrary topography.

Visual Algebra: Constructing a 1D Tower

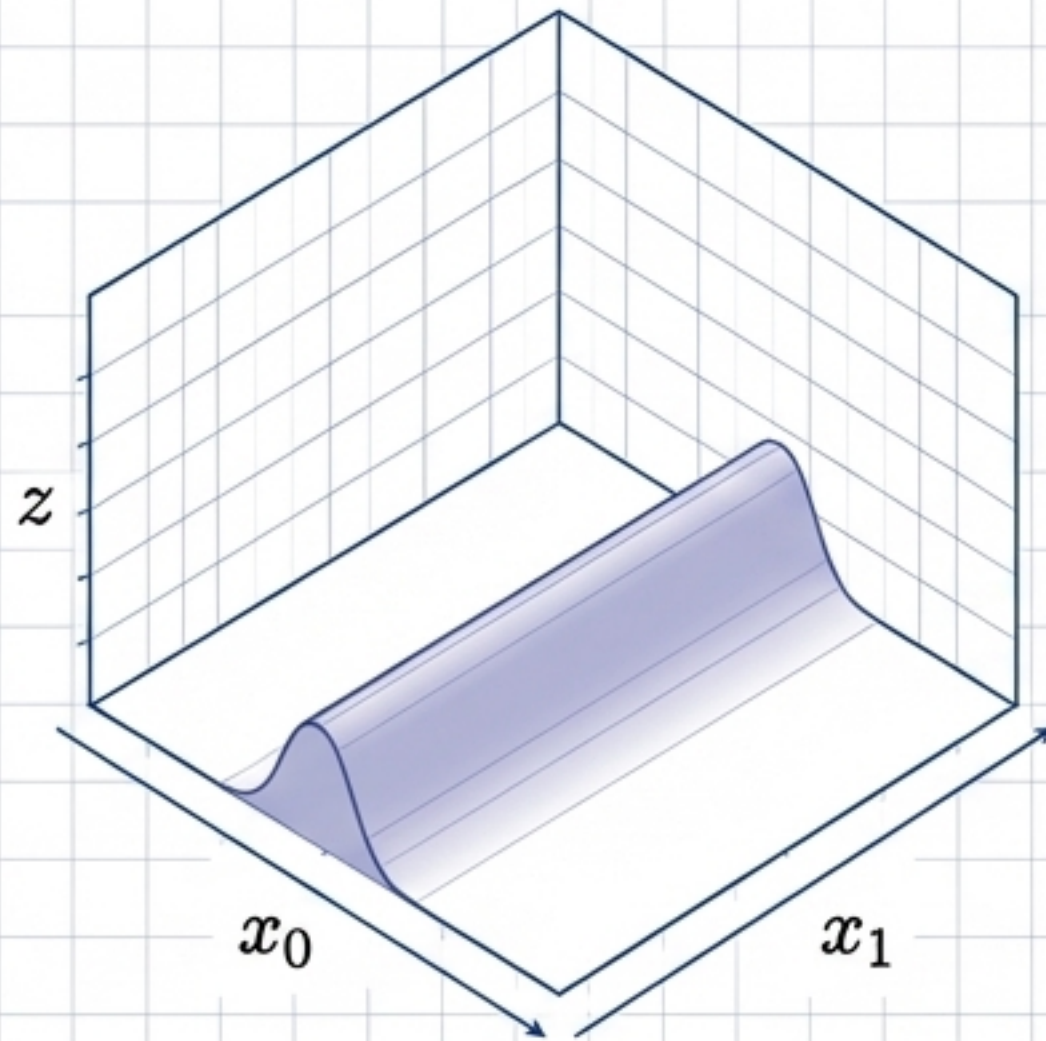


$$y = \theta(\theta(x + 5) + \theta(-x + 5) - 1.5)$$

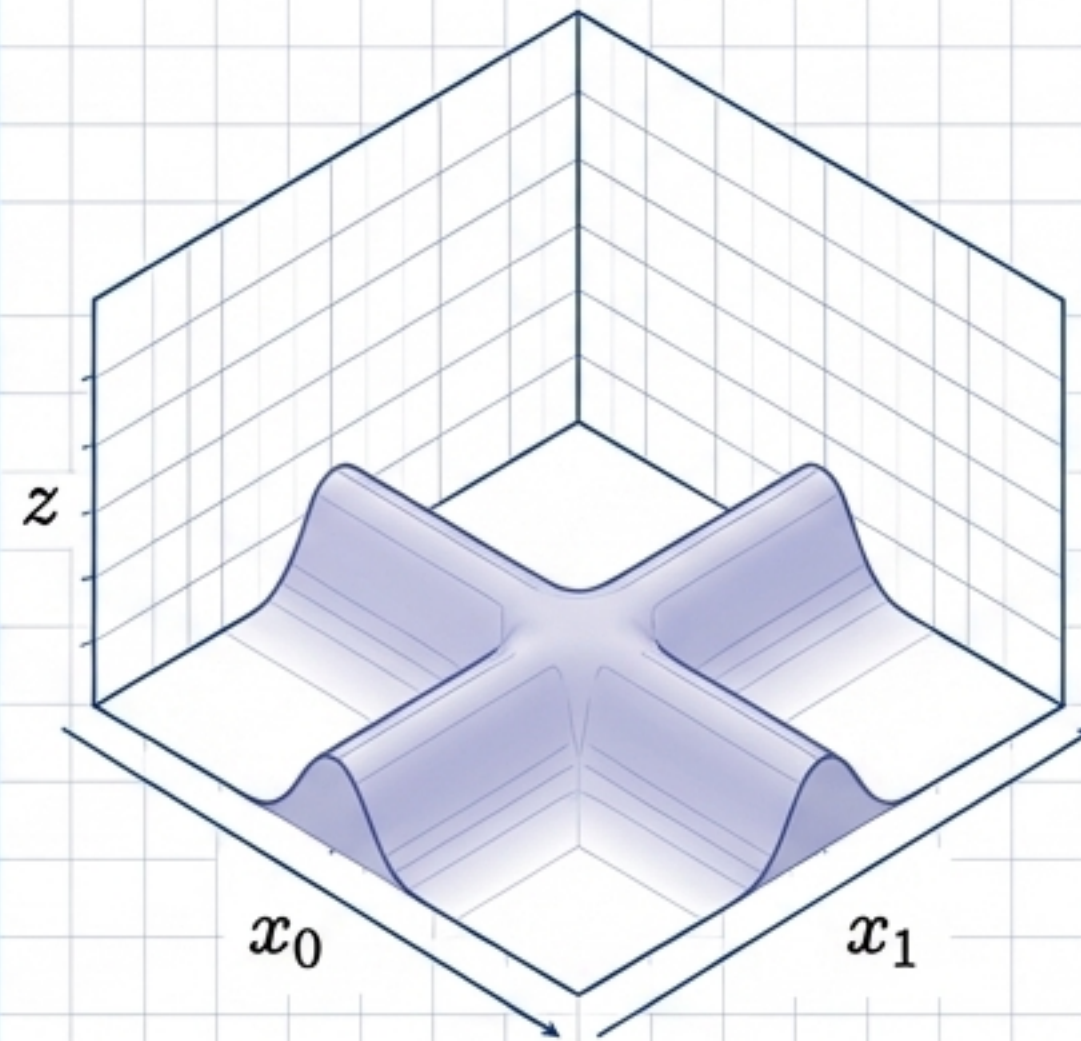
Extrapolating Dimensions: Intersecting Waves

Takeaway: By generating wave functions along individual coordinate axes and passing them through a logical AND operation, the network mathematically isolates a microscopic region in high-dimensional space.

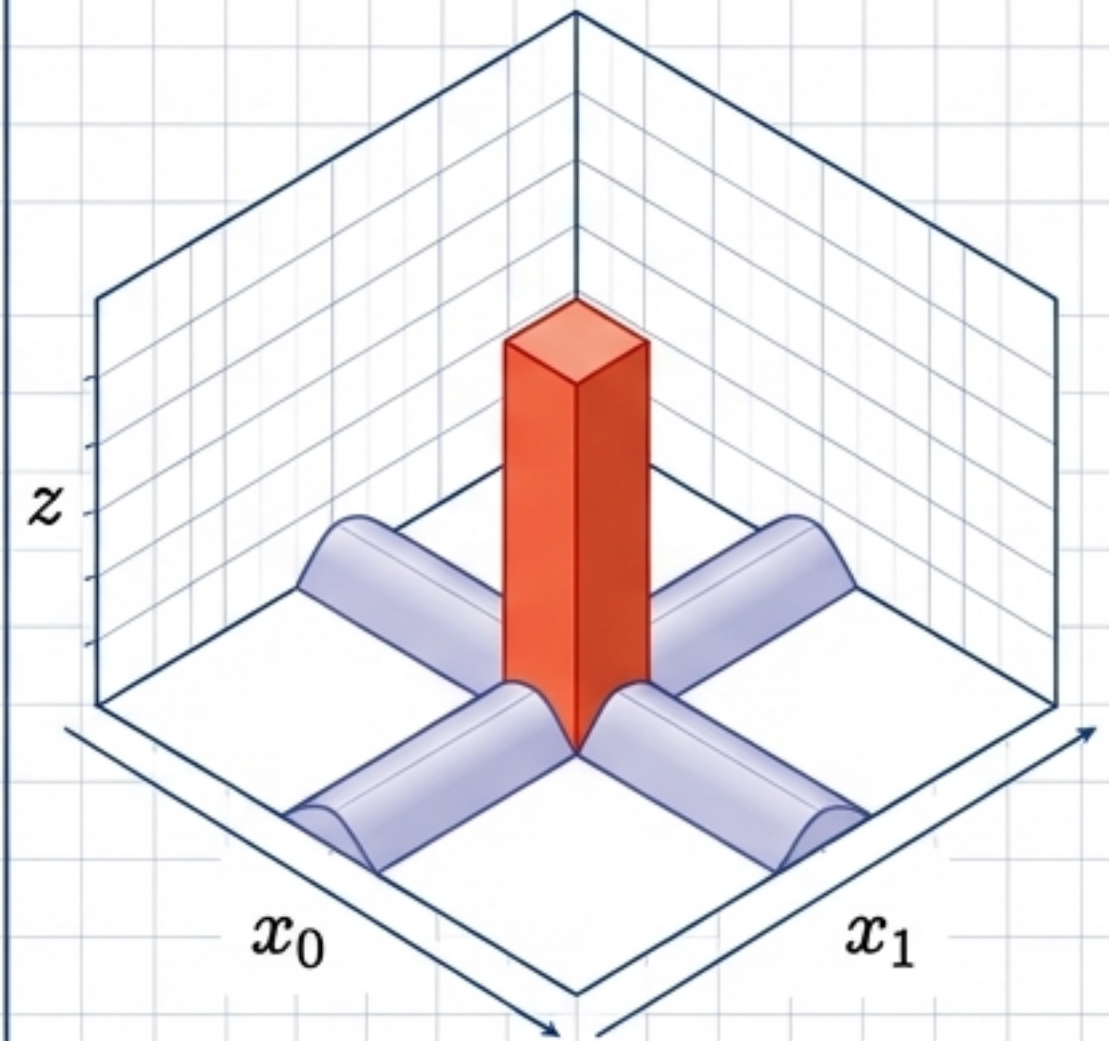
Wave x_0



Wave x_1

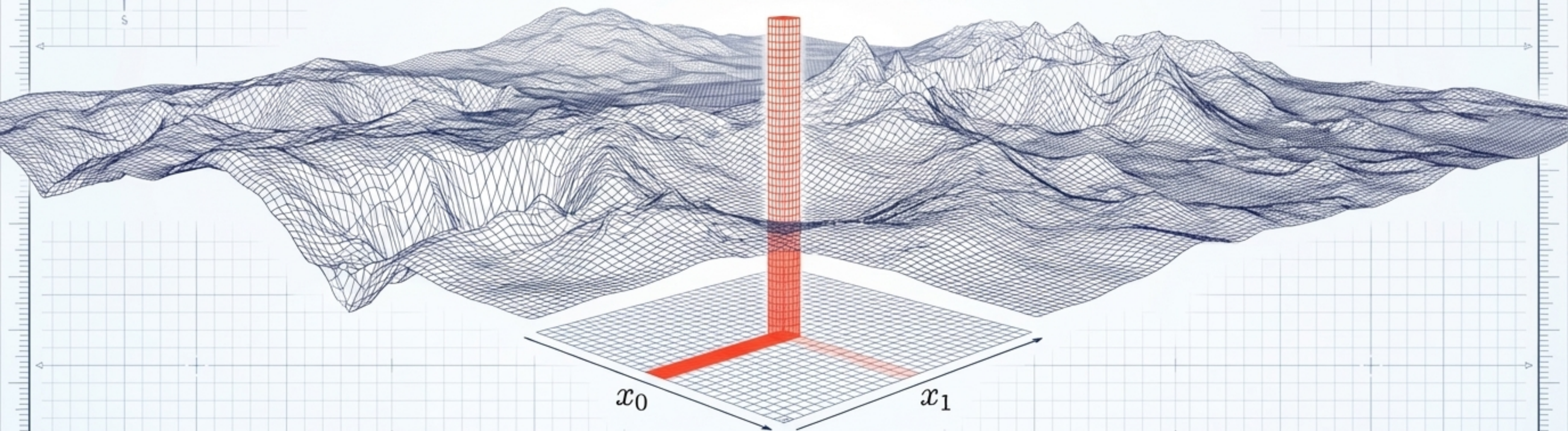


The Intersection (AND)



Synthesis: The Universal Approximator

$$y = \theta \left(\theta \left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \theta \left(\begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} \right) - 1.5 \right) - 3.5 \right)$$



This equation represents a single 2D tower built by an MLP. By adjusting the weight and bias matrices (position, width, and height) and summing thousands of these architectures, a neural network ceases to be mere code. It becomes an infinitely malleable geometric engine capable of modeling literally any phenomenon in the universe, bounded solely by the quality of its training data.